



山东大学

信息科学与工程学院

2025—2026 学年第一学期

实 验 报 告

课程名称: Java 编程技术

实验名称: 第六次实验

专 业 班 级 通信三班

学 生 学 号

学 生 姓 名

实 验 时 间 2025 年 11 月 30 日

实 验 报 告

【实验目的】

- (1) 掌握 MySQL 数据库的安装和使用
- (2) 掌握通过 JDBC 方式操作数据库的基本步骤。
- (3) 掌握增、删、改、查记录的方法。
- (4) 学习 MysqIDemo.java, 完成简单的学生信息管理

【实验具体内容】

1) 掌握 MySQL 数据库的安装和基本使用, 在本机登录 mysql 后, 利用 GRANT 授权法, 实现远程访问数据库。

安装完成 mysql 数据库后, 打开 MySQL 5.5 Command Line Client, 输入密码后开始操作数据库:

在学习后 Grant 指令,

```
GRANT 权限范围 ON 数据库.表 TO '用户名'@'远程地址' IDENTIFIED BY '密码' WITH GRANT OPTION;
```

各项参数如下:

权限范围: 常用 ALL PRIVILEGES (所有权限, 适合管理场景), 或具体权限 (如 SELECT, INSERT, UPDATE, 适合普通业务);

数据库.表: 指定用户可访问的资源, *.* 表示所有数据库 (谨慎使用), 推荐指定具体数据库 (如 my_db.* 表示仅访问 my_db 库);

用户名: 自定义远程用户 (如 remote_user);

远程地址: 允许登录的远程机器 IP / 地址, % 表示「所有远程地址」(方便测试, 生产环境建议指定具体 IP, 如 192.168.1.100);

密码: 远程用户的登录密码 (需复杂, 如 Abc123!@#);

WITH GRANT OPTION: 可选, 允许该用户给其他用户授权 (普通用户可不加)。

我们开始实验

输入 GRANT 指令:

```
GRANT ALL PRIVILEGES ON *.* TO 'remote_user'@'%' IDENTIFIED BY '123456' WITH GRANT OPTION;
```

```

Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 4
Server version: 5.5.28 MySQL Community Server (GPL)

Copyright (c) 2000, 2012, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> mysql -u root -p
-> GRANT ALL PRIVILEGES ON *.* TO 'remote_user'@'%' IDENTIFIED BY '123456' WITH GRANT OPTION;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server ver
sion for the right syntax to use near 'mysql -u root -p
GRANT ALL PRIVILEGES ON *.* TO 'remote_user'@'%' IDENTIFIED BY ' ' at line 1
mysql> \c
mysql> GRANT ALL PRIVILEGES ON *.* TO 'remote_user'@'%' IDENTIFIED BY '123456' WITH GRANT OPTION;
Query OK, 0 rows affected (0.01 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.01 sec)

mysql>

```

图 1 输入 GRANT 指令

输入 `flush privileges;` 指令后输入 `use mysql;` 与 `select user, host from mysql.user;` 观察用户是否添加成功。



```

MySQL 5.5 Command Line Client
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 6
Server version: 5.5.28 MySQL Community Server (GPL)

Copyright (c) 2000, 2012, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use mysql;
Database changed
mysql> select user, host from mysql.user;
+-----+-----+
| user      | host      |
+-----+-----+
| remote_user | %         |
| root       | %         |
| root       | localhost |
+-----+-----+
3 rows in set (0.00 sec)

mysql>

```

图 2 观察用户权限

我们可以发现，这里的远程操作用户已经添加成功。

接下来，我们采用 SQLyog 测试远程连接：



图 3 实验 SQLyog 测试连接

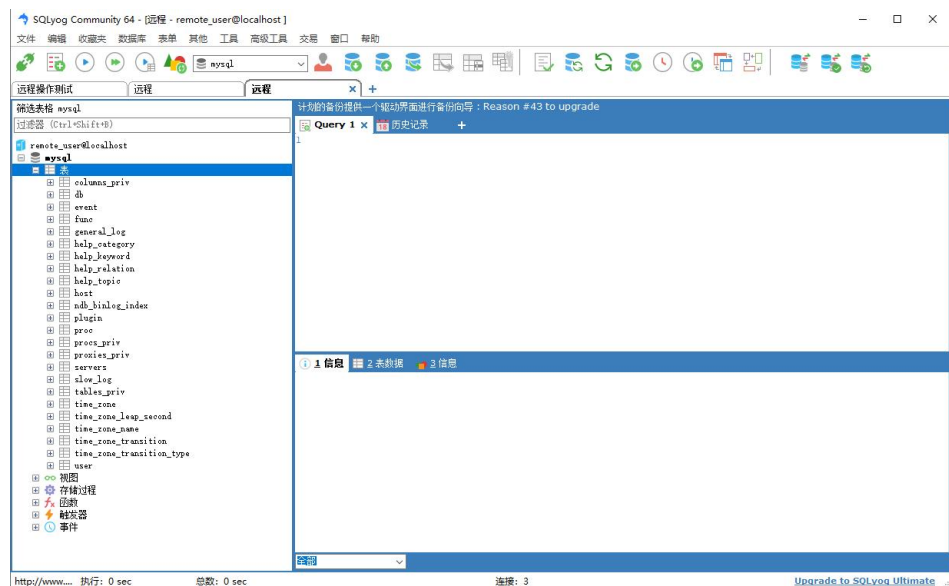


图 4 远程操作界面

因此，我们便可以在以上的界面中进行远程数据库操作了。

2) 学习 MysqIDemo.java, 熟悉 GUI 的同学可以编写一个简单的学生信息管理系统界面。如果不熟悉 GUI, 可以建立几个 button 来实现以下功能:

1) 实验源程序

```
package javaJDBC;

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
```

```

import java.awt.event.ActionListener;
import java.sql.*;
public class StudentManager extends JFrame implements ActionListener {
    private static final long serialVersionUID = 1L;
    //数据库配置

    private static final String DRIVER = "com.mysql.jdbc.Driver";
    // MySQL
    private static final String URL =
"jdbc:mysql://localhost:3306/javademo?useUnicode=true&characterEncoding=UTF8&useSSL=false";
    private static final String USER = "root";
    private static final String PASS = "123456"
    //UI 组件
    private JTextField tfId; // 学号输入框
    private JTextField tfName; // 姓名输入框
    private JTextField tfAge; // 年龄输入框
    private JTextArea displayArea; // 结果显示区域
    private JButton btnSearch; // 查询按钮
    private JButton btnAdd; // 追加按钮
    private JButton btnDelete; // 删除按钮
    private JButton btnClear; // 清空输入按钮
    public static void main(String[] args) {
        // Swing 程序建议在 EDT（事件调度线程）中启动
        SwingUtilities.invokeLater(() -> new StudentManager());
    }
    // 构造方法：初始化 UI 初始化数据库表

    public StudentManager() {
        // 初始化界面
        initUI();
        // 初始化数据库表（自动创建表 + 兼容添加 age 字段）
        initDatabaseTable();
    }
    //初始化 UI 界面

    private void initUI() {
        // 窗口基本设置
        this.setTitle("学生信息管理系统");
        this.setSize(600, 450); // 适配年龄输入框，高度调整为 450
        this.setLocationRelativeTo(null); // 窗口居中
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // 关闭窗口退出程序
        this.setLayout(new BorderLayout(10, 10)); // 边框布局，间距 10
    }

```

```

        //输入面板（学号 + 姓名 + 年龄）
        JPanel inputPanel = new JPanel(new GridLayout(3, 2, 10, 10)); // 3 行 2 列,
        // 带标题边框
        inputPanel.setBorder(BorderFactory.createTitledBorder("学生信息输入"));
        // 学号
        inputPanel.add(new JLabel("学号 (NO): ", JLabel.RIGHT));
        tfId = new JTextField();
        inputPanel.add(tfId);
        // 姓名
        inputPanel.add(new JLabel("姓名 (Name): ", JLabel.RIGHT));
        tfName = new JTextField();
        inputPanel.add(tfName);
        // 年龄
        inputPanel.add(new JLabel("年龄 (Age): ", JLabel.RIGHT));
        tfAge = new JTextField();
        inputPanel.add(tfAge);
        // 输入面板容器
        JPanel topContainer = new JPanel(new BorderLayout());
        topContainer.add(inputPanel, BorderLayout.CENTER);
        topContainer.setBorder(BorderFactory.createEmptyBorder(10, 10, 0, 10));
        // 上、左、右边距 10
        this.add(topContainer, BorderLayout.NORTH); // 放在窗口北部
        // 结果显示区域
        displayArea = new JTextArea();
        // 不可编辑
        displayArea.setEditable(false);
        displayArea.setFont(new Font("Monospaced", Font.PLAIN, 14));
        JScrollPane scrollPane = new JScrollPane(displayArea);
        scrollPane.setBorder(BorderFactory.createTitledBorder("操作结果 / 查询显示"));
        // 放在窗口中部
        this.add(scrollPane, BorderLayout.CENTER);
        // 按钮面板
        JPanel btnPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 20, 10));
        // 居中布局，按钮间距 20
        btnSearch = createButton("查询（按学号）");
        btnAdd = createButton("追加");
        btnDelete = createButton("删除（按学号）");
        btnClear = createButton("清空输入");
        btnPanel.add(btnSearch);
        btnPanel.add(btnAdd);
        btnPanel.add(btnDelete);
        btnPanel.add(btnClear);
    
```

```

        this.add(btnPanel, BorderLayout.SOUTH); // 放在窗口南部
        // 显示窗口
        this.setVisible(true);
    }
    //
    private JButton createButton(String text) {
        JButton btn = new JButton(text);
        btn.addActionListener(this); // 绑定事件监听
        return btn;
    }
    // 获取数据库连接

    private Connection getConnection() throws ClassNotFoundException,
SQLException {
        Class.forName(DRIVER); // 加载驱动 (MySQL 5.5 必须显式加载)
        return DriverManager.getConnection(URL, USER, PASS); // 获取连接
    }
    /**
     * 初始化数据库表:
     * 1. 自动创建 student 表 (如果不存在)
     * 2. 自动检查并添加 age 字段 (兼容旧表, 解决 MySQL 5.5 不支持 IF NOT EXISTS 的
问题)
     */
    private void initDatabaseTable() {
        // 1. 创建 student 表的 SQL (包含 no 主键、name 非空, 适配 MySQL 5.5)
        String createTableSql = "CREATE TABLE IF NOT EXISTS student (" +
            "no CHAR(20) PRIMARY KEY, " + // 学号: 主键, 字符类型 (长度 20)
            "name VARCHAR(20) NOT NULL, " + // 姓名: 非空, 字符串类型 (长度 20)
            "age INT " + // 年龄: 整数类型 (允许为 null, 支持未填写)
            ") ENGINE=InnoDB DEFAULT CHARSET=utf8"; // 引擎 InnoDB, 编码 utf8
        // (避免中文乱码)
        try (Connection conn = getConnection();
            Statement stmt = conn.createStatement()) {
            // 执行建表语句
            stmt.executeUpdate(createTableSql);
            log(" 数据表 'student' 检查/创建成功");
            // 2. 检查 age 字段是否存在 (MySQL 5.5 不支持 ALTER TABLE ADD COLUMN IF
NOT EXISTS)
            // 通过 INFORMATION_SCHEMA 系统表查询字段是否存在
            String checkAgeColumnSql = "SELECT COLUMN_NAME FROM
INFORMATION_SCHEMA.COLUMNS " +
                "WHERE TABLE_SCHEMA = DATABASE() " + // 当前数据库 (javademo)
                "AND TABLE_NAME = 'student' " + // 表名
                "AND COLUMN_NAME = 'age'"; // 要检查的字段名

```

```

        ResultSet rs = stmt.executeQuery(checkAgeColumnSql);
        if (!rs.next()) {
            // 3. 如果 age 字段不存在, 执行添加字段
            String addAgeColumnSql = "ALTER TABLE student ADD COLUMN age INT";
            stmt.executeUpdate(addAgeColumnSql);
            log("字段 'age' 已添加到 'student' 表");
        } else {
            log("字段 'age' 已存在, 无需重复添加");
        }
        log(" 系统就绪, 可正常使用");
    } catch (ClassNotFoundException e) {
        // 驱动未导入或驱动类名错误
        log(" 错误: 找不到 MySQL 5.5 驱动!");
        JOptionPane.showMessageDialog(this,
            "驱动缺失: 请导入 mysql-connector-java-5.1.x 驱动包\n(不要使用 8.0+ 版本驱动)",
            "驱动错误", JOptionPane.ERROR_MESSAGE);
    } catch (SQLException e) {
        // 数据库连接失败、SQL 语法错误等
        log(" 数据库操作失败: " + e.getMessage());
        JOptionPane.showMessageDialog(this,
            "数据库错误: \n1. 检查 URL 是否正确(javdemo 库是否存在)\n2. 检查账号密码是否正确\n3. 检查 MySQL 服务是否启动",
            "数据库错误", JOptionPane.ERROR_MESSAGE);
    } catch (Exception e) {
        // 其他未知错误
        log(" 系统初始化失败: " + e.getMessage());
        JOptionPane.showMessageDialog(this, "未知错误: " + e.getMessage(), "
        错误", JOptionPane.ERROR_MESSAGE);
    }
}

//功能 1: 按学号查询学生信息

private void searchStudent() {
    String studentId = tfId.getText().trim(); // 获取学号(去除前后空格)
    // 校验: 学号不能为空
    if (studentId.isEmpty()) {
        JOptionPane.showMessageDialog(this, "请输入学号进行查询!", "输入提示",
        JOptionPane.WARNING_MESSAGE);
        return;
    }
}
    
```

```

        // 查询 SQL: 获取学号、姓名、年龄 (适配 age 字段)
        String sql = "SELECT no, name, age FROM student WHERE no = '" + studentId
+ "'";

        try (Connection conn = getConnection();
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(sql)) {
            if (rs.next()) {
                // 查询到结果: 获取姓名和年龄
                String name = rs.getString("name");
                int age = rs.getInt("age");
                boolean isAgeNull = rs.wasNull(); // 判断年龄是否为 null (未填写)
                // 填充到输入框
                tfName.setText(name);
                tfAge.setText(isAgeNull ? "" : String.valueOf(age)); // 年龄 null
时显示空字符串

                // 日志显示
                log(String.format("查询成功\n学号: %s\n姓名: %s\n年龄: %s",
                    studentId, name, isAgeNull ? "未填写" : age));
            } else {
                // 未查询到结果
                log("未找到学号为 '" + studentId + "' 的学生");
                // 清空输入框
                tfName.setText("");
                tfAge.setText("");
            }
        } catch (Exception e) {
            log("查询出错: " + e.getMessage());
            JOptionPane.showMessageDialog(this, "查询失败: " + e.getMessage(), "
错误", JOptionPane.ERROR_MESSAGE);
        }
    }

    // 功能 2: 追加学生信息 (学号 + 姓名 + 年龄)

    private void addStudent() {
        // 获取输入值 (去除前后空格)
        String studentId = tfId.getText().trim();
        String name = tfName.getText().trim();
        String ageStr = tfAge.getText().trim();
        // 校验 1: 学号和姓名不能为空
        if (studentId.isEmpty() || name.isEmpty()) {
            JOptionPane.showMessageDialog(this, "学号和姓名不能为空!", "输入提示",
JOptionPane.WARNING_MESSAGE);
            return;
        }
    }

```

```

// 校验 2: 年龄格式 (可选输入, 若输入则必须是 1-150 的整数)
Integer age = null;
if (!ageStr.isEmpty()) {
    try {
        age = Integer.parseInt(ageStr);
        if (age <= 0 || age > 150) {
            JOptionPane.showMessageDialog(this, "年龄必须是 1-150 的整数!",
            "输入提示", JOptionPane.WARNING_MESSAGE);
            return;
        }
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this, "年龄格式错误, 请输入整数!", "
        输入提示", JOptionPane.WARNING_MESSAGE);
        return;
    }
}

// 插入 SQL: 年龄为 null 时插入 NULL (适配 MySQL 5.5)
String sql = String.format("INSERT INTO student(no, name, age) VALUES('%s',
'%s', %s)",
        studentId, name, age == null ? "NULL" : age);
try (Connection conn = getConnection();
    Statement stmt = conn.createStatement()) {
    int affectedRows = stmt.executeUpdate(sql);
    if (affectedRows > 0) {
        log(String.format("✔ 追加成功\n 学号: %s\n 姓名: %s\n 年龄: %s",
            studentId, name, age == null ? "未填写" : age));
        // 清空输入框 (方便下次输入)
        clearInputFields();
    } else {
        log("追加失败: 未影响任何记录");
    }
} catch (SQLIntegrityConstraintViolationException e) {
    // 主键冲突 (学号已存在)
    log("追加失败: 学号 '" + studentId + "' 已存在");
    JOptionPane.showMessageDialog(this, "学号已存在, 请输入新的学号!", "
    重复提示", JOptionPane.WARNING_MESSAGE);
} catch (Exception e) {
    log("追加出错: " + e.getMessage());
    JOptionPane.showMessageDialog(this, "追加失败: " + e.getMessage(), "
    错误", JOptionPane.ERROR_MESSAGE);
}
}

```

```

//功能 3: 按学号删除学生信息

private void deleteStudent() {
    String studentId = tfId.getText().trim();
    // 校验: 学号不能为空
    if (studentId.isEmpty()) {
        JOptionPane.showMessageDialog(this, "请输入要删除的学生学号!", "输入提示", JOptionPane.WARNING_MESSAGE);
        return;
    }
    // 确认删除 (防止误操作)
    int confirm = JOptionPane.showConfirmDialog(
        this,
        "确定要删除学号为 '" + studentId + "' 的学生吗? \n 删除后数据无法恢复!",
        "确认删除",
        JOptionPane.YES_NO_OPTION
    );
    if (confirm != JOptionPane.YES_OPTION) {
        log("删除操作已取消");
        return;
    }
    // 删除 SQL
    String sql = "DELETE FROM student WHERE no = '" + studentId + "'";
    try (Connection conn = getConnection()) {
        Statement stmt = conn.createStatement();
        int affectedRows = stmt.executeUpdate(sql);
        if (affectedRows > 0) {
            log("删除成功: 学号 '" + studentId + "' 的记录已移除");
            // 清空输入框
            clearInputFields();
        } else {
            log("删除失败: 未找到学号为 '" + studentId + "' 的学生");
        }
    } catch (Exception e) {
        log("删除出错: " + e.getMessage());
        JOptionPane.showMessageDialog(this, "删除失败: " + e.getMessage(), "错误", JOptionPane.ERROR_MESSAGE);
    }
}

// 清空所有输入框

private void clearInputFields() {

```

```

        tfId.setText("");
        tfName.setText("");
        tfAge.setText("");
    }
    // 日志输出到显示区域
    private void log(String msg) {
        displayArea.append(msg + "\n-----\n");
        // 自动滚动到底部
        displayArea.setCaretPosition(displayArea.getDocument().getLength());
    }
    //按钮点击事件分发（统一处理所有按钮点击）

    @Override
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == btnSearch) {
            searchStudent(); // 查询
        } else if (e.getSource() == btnAdd) {
            addStudent(); // 追加
        } else if (e.getSource() == btnDelete) {
            deleteStudent(); // 删除
        } else if (e.getSource() == btnClear) {
            clearInputFields(); // 清空输入
        }
    }
}

```

2) 实验过程

通过学习 `MysqlDemo` 的示例，我们学会了如何使用 JDBC 驱动 `Mysql` 数据库：

```
jdbc:mysql://主机名:端口号/数据库名?参数 1=值 1&参数 2=值 2
```

该语句进行数据库连接配置，设置好了如何连接 `Mysql` 数据库，连接哪个数据库，以什么用户连接等信息。

```
Class.forName("com.mysql.jdbc.Driver");
```

该语句进行使用反射机制进行数据库驱动。在之前的实验中我们有所涉及，此处可以降低代码的耦合度，便于我们动态更改。

```
conn = DriverManager.getConnection(url);
```

通过 `DriverManager` 类，我们得以将之前的配置语句 `url` 转化为 `mysql` 连接操作。

接下来，我们可以借助 `executeUpdate` 类来实现增删，与更新操作，这也是后面我们完成实验内容的基础：

```
Statement stmt = conn.createStatement();
```

```
sql = "create table student(NO char(20),name varchar(20),primary key(NO))";
int result = stmt.executeUpdate(sql);
```

我们可以看到，我们同样使用字符串拼接 mysql 指令，通过特定的类将 mysql 指令运行，因此，我们可以对 JDBC 操作 mysql 进行一个简单的流程图概况：

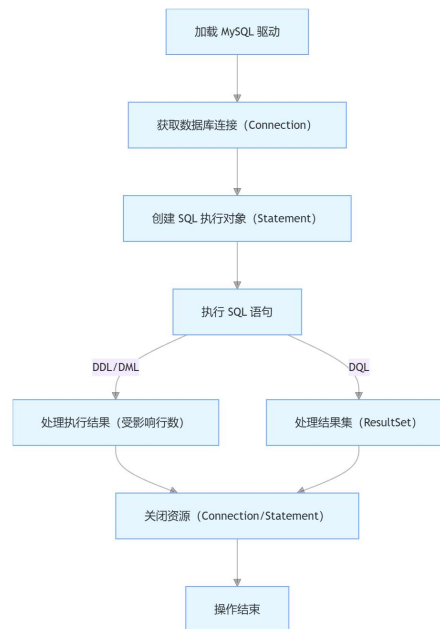


图 5 流程图概括

因此，我们可以开始对于简易的学生信息管理系统开始着手制作了：

根据之前完成大作业的经验，我们可以实现一个简易的 GUI，即加入几个监听按钮，同时为下方的添加 JtxttField 以显示我们的的查询结果。

先创建 student 表，包含学生的学号、姓名、年龄信息。

```
String createTableSql = "CREATE TABLE IF NOT EXISTS student (" +
    "no CHAR(20) PRIMARY KEY, " + // 学号：主键，字符类型（长度 20）
    "name VARCHAR(20) NOT NULL, " + // 姓名：非空，字符串类型（长度 20）
    "age INT " + // 年龄：整数类型（允许为 null，支持未填写）
    ") ENGINE=InnoDB DEFAULT CHARSET=utf8";
Statement stmt = conn.createStatement(); {

    // 执行建表语句
    stmt.executeUpdate(createTableSql);
}
```

如此，我们便可以成功建里一个包含 age 字段的表，接下来，便是分别实现管理系统的各个功能：

① 点击“查询”按钮，根据学号，可以查询到学生的姓名和年龄；

我们可以将接收到的 Jtext 的结果使用 getText 指令获取，然后使用字符串拼接成 sql 语句，使用 executeQuery 方法实现管理系统的查询。

这里使用的 sql 语句为：

```
"SELECT no, name, age FROM student WHERE no = '" + studentId + "'";
```

如上，同时我们需要对于其进行异常处理，如上便可以查询功能的构建：

```
private void searchStudent() {
    String studentId = tfId.getText().trim(); // 获取学号（去除前后空格）

    // 校验：学号不能为空
    if (studentId.isEmpty()) {
        JOptionPane.showMessageDialog(this, "请输入学号进行查询！", "输入提示",
        JOptionPane.WARNING_MESSAGE);
        return;
    }
    // 查询 SQL：获取学号、姓名、年龄（适配 age 字段）
    String sql = "SELECT no, name, age FROM student WHERE no = '" + studentId
    + "'";
    try (Connection conn = getConnection();
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(sql)) {
        if (rs.next()) {
            // 查询到结果：获取姓名和年龄
            String name = rs.getString("name");
            int age = rs.getInt("age");
            boolean isAgeNull = rs.wasNull(); // 判断年龄是否为 null（未填写）
            // 填充到输入框
            tfName.setText(name);
            tfAge.setText(isAgeNull ? "" : String.valueOf(age)); // 年龄 null
            // 日志显示
            log(String.format("查询成功\n学号: %s\n姓名: %s\n年龄: %s",
                studentId, name, isAgeNull ? "未填写" : age));
        } else {
            // 未查询到结果
            log("未找到学号为 '" + studentId + "' 的学生");
            // 清空输入框
            tfName.setText("");
            tfAge.setText("");
        }
    } catch (Exception e) {
        log("查询出错: " + e.getMessage());
    }
}
```

```
JOptionPane.showMessageDialog(this, "查询失败: " + e.getMessage(), "
错误", JOptionPane.ERROR_MESSAGE);
}
}
```

② 点击“追加”按钮，根据给定的学生学号、姓名、年龄，在表中追加一行信息；

接着，我们开始进行追加功能的构建：

与上相同的是，我们要进行的操作是一样的，仅是 sql 语句发生了变化：

```
INSERT INTO student(no, name, age) VALUES('%s', '%s', %s)
```

这里就不再赘述。

③ 点击“删除”按钮，利用给定的学生学号，可以从表中删除该学生的信息；

这里使用的 sql 语句为：

```
DELETE FROM student WHERE no = + studentId + ""
```

代码实现同上。

运行实验代码，我们观察一下实验效果：



图 6 追加功能验证



图 7 查询功能验证



图 8 删除功能的验证

可以发现，我们学生信息管理系统的基本功能已经构建完成。

【实验心得】

通过本次实验，我深入掌握了 Java 程序通过 JDBC 技术操作 MySQL 数据库的完整流程。实验过程中，我不仅学会了数据库的基本操作，还进行了一个简易学生信息管理系统实战。

学习 JDBC 驱动 Mysql 时，我学到了如何驱动 Mysql 库，最让我印象深刻的是我们对于驱动库的动态加载，我们是通过反射机制，即 `Class.forName("com.mysql.jdbc.Driver");` 最后利

用 `DriverManager.getConnection(url)` 建立实际连接。这种分层设计让我理解了 Java 数据库连接的工作原理。

然后，我们使用 `statement` 语句来实现对于数据库的增删查改功能，这个操作确实非常简单，但是在深入了解后，我得知这样的操作并不安全，会有 SQL 注入的风险——SQL 注入是攻击者利用程序输入校验的疏忽，在输入参数中插入恶意的 SQL 代码片段，从而篡改原始 SQL 语句的语义，达到非法查询、篡改甚至删除数据的目的。如用户在学号输入框中不输入正常的学号，而是输入：`' OR '1' = '1` 那么，最终拼接而成的 SQL 语句将变为：

```
SELECT no, name, age FROM student WHERE no = '' OR '1' = '1'
```

这个语句的 WHERE 条件变成了永远为真，导致查询出 `student` 表中的所有学生记录，而非指定的某一条。

因此在实际应用中，更常用的是 `PreparedStatement` 来替代 `Statement`，因为它能够预编译 SQL 语句，使用参数化查询，有效防止 SQL 注入攻击，同时还能提高执行效率。

最后通过构建学生信息管理系统，我布局复习了之前 GUI 的知识，同时还实践了这些 sql 理论知识：使用 `executeUpdate` 实现数据的增删改操作，使用 `executeQuery` 实现查询功能，并通过 `ResultSet` 处理返回的结果集。特别是在实现查询、追加、删除功能时，我深刻体会到 SQL 语句构造与 Java 代码的紧密结合。